



Documentação Técnica — Baycad RH

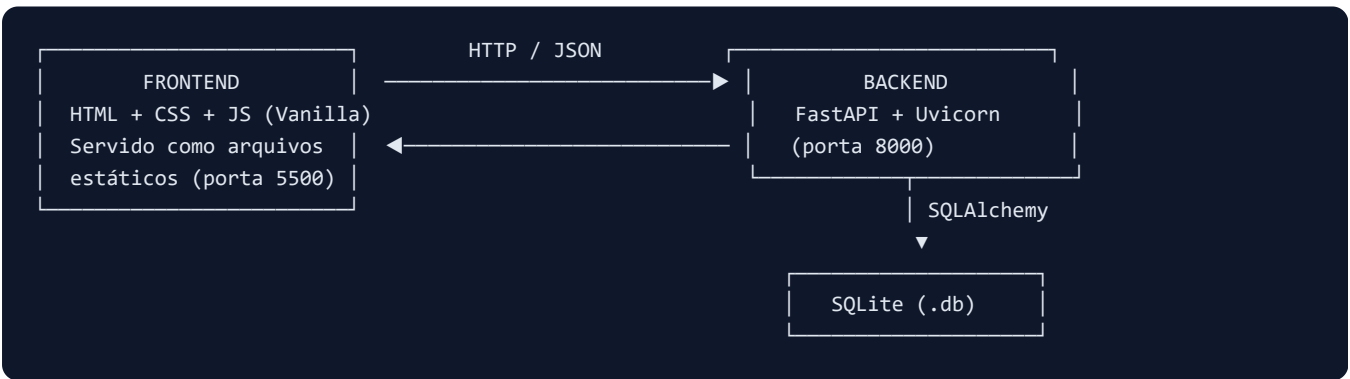
Sistema web de Gestão de Recursos Humanos (RH) composto por um frontend estático (HTML/CSS/JS) integrado a uma API REST em **FastAPI** com persistência em **SQLite**.

Documento de referência técnica. Para o passo a passo de uso e os casos de teste, consulte o [ROTEIRO.md](#).

1. Visão Geral

Item	Descrição
Nome	Baycad RH · v1.0
Tipo	Aplicação web cliente-servidor
Domínio	Gestão de funcionários (cadastro, consulta, edição, exclusão)
Autenticação	Login por CPF + senha, com hash BCrypt
Banco	SQLite (<code>python/baycad_rh.db</code>)

2. Arquitetura



- **Comunicação:** o frontend chama a API via `fetch`, sempre em JSON.
- **CORS:** liberado para todas as origens (`allow_origins=["*"]`) — adequado para ambiente de teste; em produção deve ser restringido.
- **Base da API:** definida uma única vez em `js/utils.js` → `const API_URL = "http://127.0.0.1:8000/api";`

3. Estrutura de Pastas

```
.
├── html/
│   ├── css/
│   │   └── style.css          # Estilo global (tema, sidebar, modais, formulários)
│   ├── login.html           # Tela de login + modal "Solicitar acesso"
│   ├── index.html           # Dashboard (métricas por departamento)
│   ├── funcionarios.html     # Listagem + modais de edição e exclusão
│   └── cadastro.html        # Formulário de cadastro
├── js/
│   ├── utils.js             # API_URL, navegação, logout (sair), toast, formatDate
│   ├── login.js             # Autenticação + solicitação de acesso (demo)
│   ├── cadastro.js          # Validação e POST de novo funcionário
│   ├── funcionarios.js      # Listagem, busca, edição (PUT) e exclusão (DELETE)
│   ├── dashboard.js         # Consumo do endpoint de métricas
│   └── database.js           # (legado) helpers de localStorage, não usado no fluxo atual
├── python/
│   ├── main.py              # Rotas da API (FastAPI)
│   ├── models.py            # Modelo ORM Funcionario + hash de senha
│   ├── schemas.py           # Schemas Pydantic (Create, Update, Response)
│   ├── database.py          # Engine, sessão e get_db
│   └── baycad_rh.db          # Banco SQLite (gerado automaticamente)
├── requirements.txt          # Dependências Python
├── netlify.toml              # Configuração de deploy do frontend
├── README.md                 # Apresentação do projeto
├── DOCUMENTACAO.md          # Este documento
└── ROTEIRO.md                # Roteiro de uso e de testes
```

4. Modelo de Dados

Tabela `funcionarios` (definida em `python/models.py`):

Campo	Tipo	Restrições	Observação
<code>id</code>	Integer	PK, autoincremento	Identificador único
<code>nome</code>	String	NOT NULL	Nome completo
<code>cpf</code>	String	UNIQUE, NOT NULL, indexado	Armazenado somente com dígitos
<code>departamento</code>	String	NOT NULL	TI, RH, Financeiro, etc.
<code>nivel</code>	String	NULL	Júnior, Pleno, Sênior, etc.
<code>data_admissao</code>	String	NOT NULL	Formato <code>YYYY-MM-DD</code>
<code>senha_hash</code>	String	NOT NULL	Hash BCrypt (nunca texto puro)

Segurança da senha (`models.py`): - `gerar_hash(senha)` — gera o hash com Passlib + BCrypt no cadastro/edição. - `verificar_senha(senha)` — compara a senha informada com o hash no login.

5. API REST

Prefixo de todas as rotas: `/api`. Documentação interativa automática em `http://127.0.0.1:8000/docs` (Swagger UI).

5.1 `POST /api/login`

Autentica um funcionário.

```
// Request
{ "cpf": "529.982.247-25", "senha": "senha123" }

// 200 OK
{ "id": 1, "nome": "Maria", "departamento": "TI", "nivel": "Pleno" }

// 401 Unauthorized
{ "detail": "CPF ou senha inválidos." }
```

O CPF é normalizado (apenas dígitos) antes da consulta.

5.2 `POST /api/funcionarios`

Cadastra um novo funcionário. Body validado por `FuncionarioCreate`.

```
// Request
{
  "nome": "Maria Silva",
  "cpf": "529.982.247-25",
  "departamento": "TI",
  "nivel": "Pleno",
  "data_admissao": "2024-01-15",
  "senha": "senha123"
}
// 201 Created → FuncionarioResponse
// 400 Bad Request → { "detail": "CPF já cadastrado." }
```

5.3 `GET /api/funcionarios`

Lista funcionários, com filtros opcionais.

Query param	Descrição
<code>busca</code>	Texto: se numérico, filtra por CPF; senão, por nome
<code>dept</code>	Filtra por departamento exato

Retorna `200 OK` com lista de `FuncionarioResponse`.

5.4 `PUT /api/funcionarios/{func_id}`

Atualiza um funcionário existente (edição). Body validado por `FuncionarioUpdate` — todos os campos são opcionais e apenas os enviados são alterados (`exclude_unset`).

- Se `cpf` for enviado, valida que não pertence a **outro** funcionário.
- Se `senha` for enviada, regenera o hash; caso contrário, mantém a senha atual.

```
// 200 OK → FuncionarioResponse
// 400 → { "detail": "CPF já cadastrado." }
// 404 → { "detail": "Funcionário não encontrado." }
```

5.5 DELETE /api/funcionarios/{func_id}

Exclui um funcionário. `204 No Content` em sucesso; `404` se não existir.

5.6 GET /api/dashboard

Retorna métricas consolidadas.

```
// 200 OK
{ "total": 12, "por_departamento": { "TI": 5, "RH": 3, "Financeiro": 4 } }
```

6. Frontend — Telas e Responsabilidades

Tela / Arquivo	Função principal	JS associado
<code>login.html</code>	Login por CPF/senha + Solicitar acesso (modal demonstrativo)	<code>login.js</code>
<code>index.html</code>	Dashboard com total e distribuição por departamento	<code>dashboard.js</code>
<code>funcionarios.html</code>	Listagem, busca, filtro, edição e exclusão	<code>funcionarios.js</code>
<code>cadastro.html</code>	Cadastro de novo funcionário	<code>cadastro.js</code>

Recursos compartilhados (`utils.js`): - `sair()` — logout: limpa `localStorage` e redireciona para `login.html` . - `toast(msg, tipo)` — notificação visual temporária. - `formatDate(d)` — converte `YYYY-MM-DD` em `DD/MM/YYYY` .

Após o login bem-sucedido, os dados do usuário são guardados em `localStorage["usuario_logado"]` (sessão simples do lado do cliente). O botão **Sair**, presente no rodapé da barra lateral de todas as telas internas, remove essa chave.

7. Funcionalidades Recentes

Funcionalidade	Onde	Resumo
Correção do login	<code>js/login.js</code>	Removida a leitura duplicada de <code>response.json()</code> , que travava o login mesmo com credenciais corretas.
Edição de funcionário	<code>PUT /api/funcionarios/{id}</code> , <code>funcionarios.js</code> , <code>funcionarios.html</code>	Modal de edição com atualização parcial e validação de CPF duplicado; senha opcional.
Logout (Sair)	<code>utils.js</code> + sidebars	Botão para encerrar a sessão em todas as telas internas.
Solicitar acesso	<code>login.html</code> , <code>login.js</code>	Modal de solicitação de acesso demonstrativo (sem backend): valida campos e exibe tela de confirmação.

8. Tecnologias e Dependências

Backend (`requirements.txt`): FastAPI, Uvicorn, SQLAlchemy, Pydantic v2, Passlib, BCrypt, Starlette, AnyIO, entre outras.

Frontend: HTML5, CSS3 e JavaScript puro (sem frameworks ou build).

9. Como Executar Localmente

Requisitos: Python 3.10+ instalado.

```
# 1. Criar e ativar ambiente virtual (Windows / PowerShell)
python -m venv .venv
.\venv\Scripts\Activate.ps1

# 2. Instalar dependências
pip install -r requirements.txt

# 3. Subir o backend (a partir da pasta python/)
python -m uvicorn main:app --reload --app-dir python
#   → API em http://127.0.0.1:8000 | Docs em http://127.0.0.1:8000/docs

# 4. Em outro terminal, servir o frontend
python -m http.server 5500
#   → App em http://127.0.0.1:5500/html/login.html
```

Observação: o frontend usa caminhos relativos (`../js/...`), portanto deve ser servido a partir da **raiz do projeto** (e acessado em `/html/...`), e não abrindo o arquivo HTML diretamente via `file://`.

10. Limitações Conhecidas

- **Sessão apenas no cliente:** o login não emite token; as páginas internas não bloqueiam acesso direto por URL. Para uso real, adotar autenticação por token/JWT.
- **CORS aberto:** `allow_origins=["*"]` deve ser restringido em produção.
- **Solicitação de acesso:** é apenas demonstrativa — não persiste nem notifica.
- `js/database.js` : código legado baseado em `localStorage` , não utilizado no fluxo atual (os dados vêm da API).